

Ensemble Learning

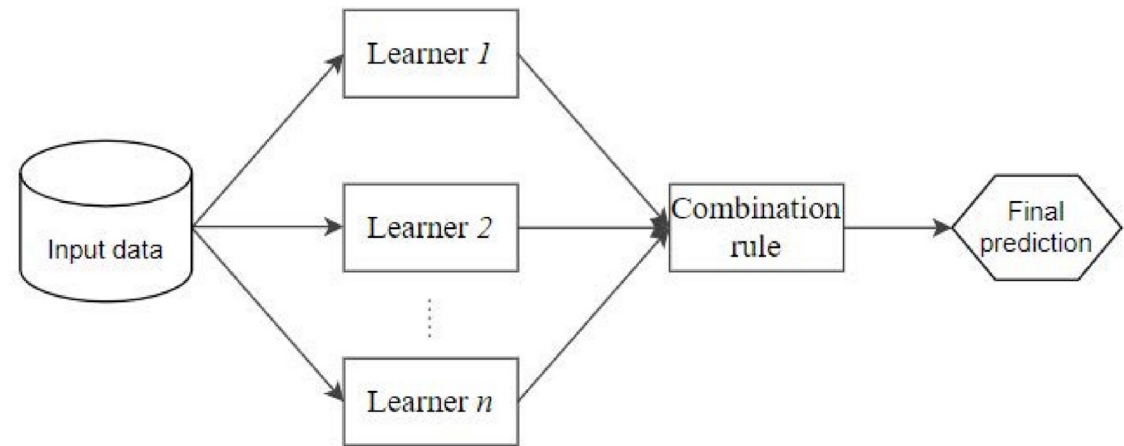
Bagging, Boosting, Gradient Boosting, XGBoost

Ensemble Method

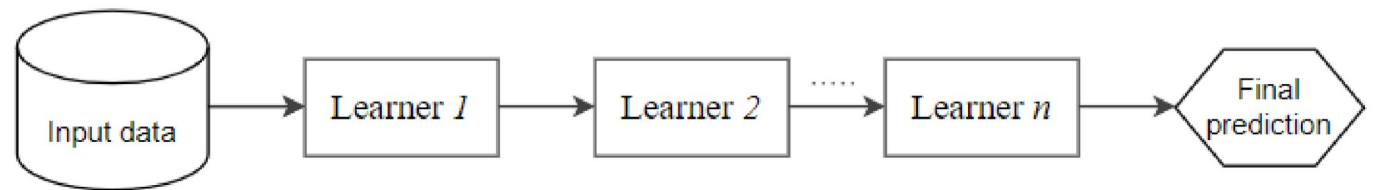
- Why train only one predictor when we can train many?
 - Training many predictor is computationally expensive
- But what if we combine many predictors?
- Ensemble of many predictors
 - Combination of many predictors
 - Weighted, Unweighted, Majority Voting, etc.
 - Same or different predictors

Ensemble Method

- Parallel Ensemble Methods
 - Bagging



- Sequential Ensemble Methods
 - Boosting



Bagging (Bootstrap Aggregating)

- Bootstrap

- Randomly draw S_j samples from S samples with replacements

- Aggregate

- Bootstrap T times
 - Train a predictor on each bootstrap
 - Inference
 - Combine the prediction of each predictor
 - Regression: Average
 - Classification: Majority Voting

Bagging (Bootstrap Aggregating)

Algorithm 4 Bagging

Input: training data $S = (x_1, y_1), \dots, (x_2, y_2), \dots, (x_n, y_n)$

Base ML algorithm L

The number of base learners T .

Procedure:

for $t = 1, \dots, T$:

1) Generate a bootstrap sample S_j from the input data S

2) Fit a base learner h_j using S_j , i.e. $h_j = L(S_j)$

end for

Output: Combine the outputs of the base learners, $H(x) = \text{mode}(h_1(x), \dots, h_T(x))$

Bagging (Bootstrap Aggregating)

- Why sample with replacement?
- Several intuitions
 - Duplicate entries get more importance in the model
 - Each predictor focuses on different regions
 - Increased diversity among the predictors
 - Etc.

Bagging (Bootstrap Aggregating)

- But how to be sure that we have enough diversity in the dataset?
 - For example, all the sampled data point may be the same
- If bootstrap size is same as the size of the dataset ($S_j = S$)
 - Bootstraps covers 63% unique samples on average
 - The probability that a point is never selecting in a bootstrap,
$$\left(1 - \frac{1}{S}\right)^S \approx e^{-1} \approx 0.368$$
 - The probability of a sample being selected is approximately 0.63

Bagging (Bootstrap Aggregating)

◦ Why does bagging improve performance?

- Recall bias-variance tradeoff for linear regression

$$\text{MSE}(x) = \underbrace{\sigma^2}_{\text{Noise}} + \underbrace{(h_{\text{avg}}(x) - h^*(x))^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}_{\mathcal{D}} [(h(x) - h_{\text{avg}}(x))^2]}_{\text{Variance}}$$


$$\sigma_h^2 = \mathbb{E}_{\mathcal{D}} [(h(x) - h_{\text{avg}}(x))^2]$$

- Instead of $h(x)$ we now have,

$$h_{\text{bag}}(x) = \frac{1}{M} \sum_{m=1}^M h_m(x)$$

Bagging (Bootstrap Aggregating)

- Let's first focus only on the last term,

$$\text{MSE}_{\text{bag}}(x) = \sigma^2 + (h_{\text{avg, bag}}(x) - h^*(x))^2 + \mathbb{E}_{\mathcal{D}} [(h_{\text{bag}}(x) - h_{\text{avg, bag}}(x))^2]$$

$$\begin{aligned}\mathbb{E}_{\mathcal{D}} [(h_{\text{bag}}(x) - h_{\text{avg}}(x))^2] &= \text{Var} \left(\frac{1}{M} \sum_{m=1}^M h_m(x) \right) \\ &= \frac{1}{M^2} \sum_{i,j} \text{Cov}(h_i(x), h_j(x))\end{aligned}$$

- If the predictors are weakly correlated, say $\text{Cov}(h_i(x), h_j(x)) = \rho$

$$\text{Var}(h_{\text{bag}}(x)) = \sigma_h^2 \left(\rho + \frac{1 - \rho}{M} \right) \begin{cases} \rho = 0 \rightarrow \frac{\sigma_h^2}{M} \\ \rho = 1 \rightarrow \sigma_h^2 \end{cases}$$

Bagging (Bootstrap Aggregating)

- Bagging reduces variance

$$\text{MSE}_{\text{bag}}(x) = \sigma^2 + (h_{\text{avg, bag}}(x) - h^*(x))^2 + \mathbb{E}_{\mathcal{D}} [(h_{\text{bag}}(x) - h_{\text{avg, bag}}(x))^2]$$

- What about bias?
 - Since the class of predictors is same as the single predictor, assuming each bootstraps have the same empirical distribution as the training data we can show that,

$$h_{\text{avg, bag}}(x) = h_{\text{avg}}(x)$$

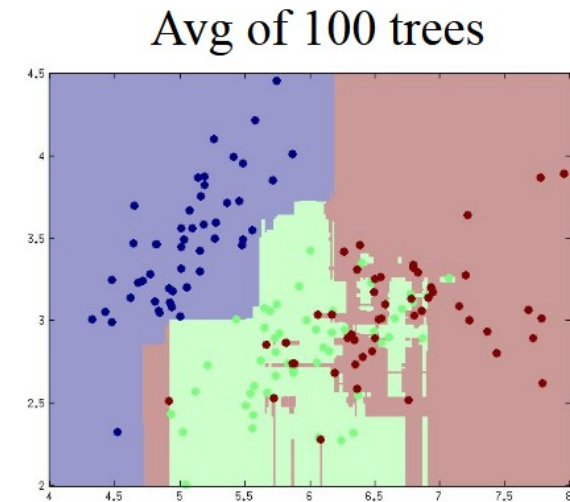
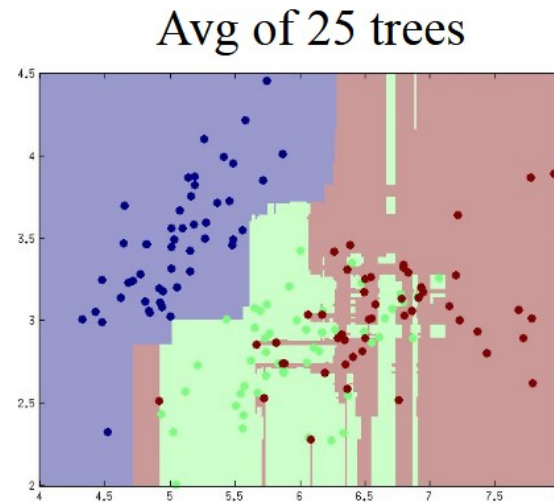
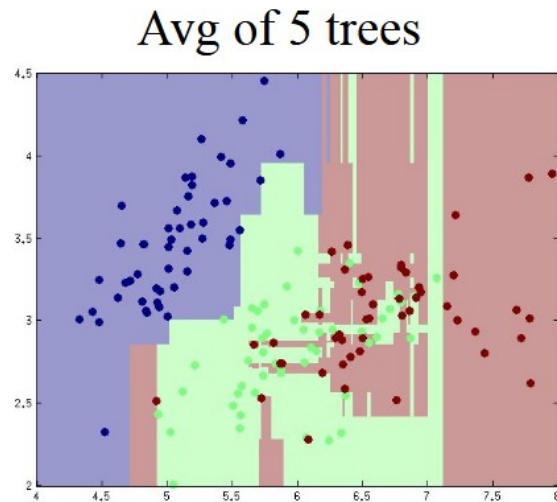
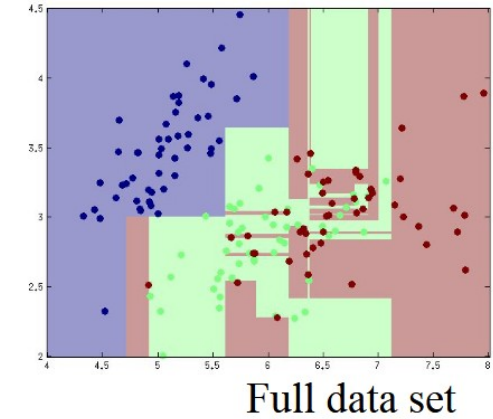
- Bias remains same

Random Forest

- When lots of data, we typically get the same predictor
- Averaging doesn't help a lot
 - High $\rho \approx 1$
- Introduce extra variation in learner
 - At each step of training, only allow a subset of features
 - Enforces diversity (“best” feature not available)
 - Average over these learners (majority vote)

Bagging (Bootstrap Aggregating)

- Visualization



Random Forest

Algorithm 5 Random Forest Algorithm

Input: training data $S = (x_1, y_1), \dots, (x_2, y_2), \dots, (x_n, y_n)$,
 p attributes and class variables.

The number of trees T in the forest

The number of class labels C

Procedure:

for $t = 1, \dots, T$:

- 1) Generate a bootstrap sample S_j from the input data S
- 2) Fit a base learner h_j using S_j , and for a given node n ,
 - (A) Randomly select k attributes. Usually $k \approx \sqrt{p}$ for classification tasks.
 - (B) Compute the best split features using the randomly selected feature subset.
 - (C) Split the node using the optimal split features obtained in step B.Repeat A-C until the stopping criteria are achieved.

- 3) Repeat steps 1 and 2 for T times to build a forest of T trees.

end for

Output: Combine the outputs of the various trees. For a given test sample x , the final predicted class label from the T trees is:

$$H_T(x) = \operatorname{argmax}_j \sum_{k=1}^K I(h_k(x) = j, \text{ for } j = 1, \dots, C)$$

Algorithm 5 Random Forest Algorithm

Input: training data $S = (x_1, y_1), \dots, (x_2, y_2), \dots, (x_n, y_n)$,
 p attributes and class variables.

The number of trees T in the forest

The number of class labels C

Procedure:

for $t = 1, \dots, T$:

- 1) Generate a bootstrap sample S_j from the input data S

Random Forest

Algorithm 5 Random Forest Algorithm

Input: training data $S = (x_1, y_1), \dots, (x_2, y_2), \dots, (x_n, y_n)$,
 p attributes and class variables.

The number of trees T in the forest

The number of class labels C

Procedure:

for $t = 1, \dots, T$:

1) Generate a bootstrap sample S_j from the input data S

2) Fit a base learner h_j using S_j , and for a given node n ,

(A) Randomly select k attributes. Usually $k \approx \sqrt{p}$ for classification tasks.

(B) Compute the best split features using the randomly selected feature subset.

(C) Split the node using the optimal split features obtained in step B.

Repeat A-C until the stopping criteria are achieved.

3) Repeat steps 1 and 2 for T times to build a forest of T trees.

end for

Output: Combine the outputs of the various trees. For a given test sample x , the final predicted class label from the T trees is:

$$H_T(x) = \operatorname{argmax}_j \sum_{k=1}^K I(h_k(x) = j, \text{ for } j = 1, \dots, C)$$

2) Fit a base learner h_j using S_j , and for a given node n ,

(A) Randomly select k attributes. Usually $k \approx \sqrt{p}$ for classification tasks.

(B) Compute the best split features using the randomly selected feature subset.

(C) Split the node using the optimal split features obtained in step B.

Repeat A-C until the stopping criteria are achieved.

3) Repeat steps 1 and 2 for T times to build a forest of T trees.

end for

Output: Combine the outputs of the various trees. For a given test sample x , the final predicted class label from the T trees is:

$$H_T(x) = \operatorname{argmax}_j \sum_{k=1}^K I(h_k(x) = j, \text{ for } j = 1, \dots, C)$$

Boosting

- Focus new experts on examples that others get wrong
 - Train experts sequentially
 - Errors of early experts indicate the “hard” examples
 - Focus later classifiers on getting these examples right
 - Combine the whole set in the end
 - Convert many “weak” learners into a complex classifier

Adaboost

Algorithm 1 AdaBoost Algorithm

Require: Training set $\{(x_1, y_1), \dots, (x_n, y_n)\}$, $y_i \in \{-1, +1\}$, Number of weak learners T

1: Initialize sample weights: $w_i = \frac{1}{n}$, $i = 1, \dots, n$

2: **for** $t = 1$ to T **do**

3: Train weak learner $h_t(x)$ using weights w_i

4: Compute weighted error:

$$\epsilon_t = \frac{\sum_{i=1}^n w_i \cdot \mathbf{1}(h_t(x_i) \neq y_i)}{\sum_{i=1}^n w_i}$$

5: Compute learner weight:

$$\alpha_t = \frac{1}{2} \ln \frac{1 - \epsilon_t}{\epsilon_t}$$

6: Update sample weights:

$$w_i \leftarrow w_i \cdot \exp(-\alpha_t y_i h_t(x_i)), \quad i = 1, \dots, n$$

7: Normalize weights:

$$w_i \leftarrow \frac{w_i}{\sum_{j=1}^n w_j}, \quad i = 1, \dots, n$$

8: **end for**

9: Output final strong classifier:

$$H(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(x) \right)$$

Adaboost

- **Training exponential loss with Adaboost strictly decreases at every iteration.**

- Proof:

- Ensemble at time t ,
$$F_t(x) = \sum_{m=1}^t \alpha_m h_m(x)$$

- Exponential Loss,
$$L_t = \sum_{i=1}^n \exp(-y_i F_t(x_i))$$

- Update Rule,
$$F_{t+1}(x) = F_t(x) + \alpha h(x)$$

Adaboost

6: Update sample weights:

$$w_i \leftarrow w_i \cdot \exp(-\alpha_t y_i h_t(x_i)), \quad i = 1, \dots, n$$

- **Training exponential loss with Adaboost strictly decreases at every iteration.**

- Proof:

- Loss after step $t+1$,
$$\begin{aligned} L_{t+1} &= \sum_i \exp(-y_i(F_t(x_i) + \alpha h(x_i))) \\ &= \sum_i \exp(-y_i F_t(x_i)) \cdot \exp(-\alpha y_i h(x_i)) \\ &= \sum_i w_i \exp(-\alpha y_i h(x_i)) \end{aligned}$$

Adaboost

- **Training exponential loss with Adaboost strictly decreases at every iteration.**

- Proof:

- Split loss into two parts, $L_{t+1} = e^{-\alpha} \sum_{i \in \text{correct}} w_i + e^{\alpha} \sum_{i \in \text{wrong}} w_i$

- Let,

$$W = \sum_i w_i \quad \varepsilon = \frac{\sum_{i \in \text{wrong}} w_i}{W}$$

$$L_{t+1} = W [(1 - \varepsilon)e^{-\alpha} + \varepsilon e^{\alpha}]$$

Adaboost

- **Training exponential loss with Adaboost strictly decreases at every iteration.**

- **Proof:**

- Minimize wrt α , $\frac{d}{d\alpha} [(1 - \varepsilon)e^{-\alpha} + \varepsilon e^{\alpha}] = 0$

$$-(1 - \varepsilon)e^{-\alpha} + \varepsilon e^{\alpha} = 0$$

$$\varepsilon e^{\alpha} = (1 - \varepsilon)e^{-\alpha}$$

$$e^{2\alpha} = \frac{1 - \varepsilon}{\varepsilon}$$

$$\alpha = \frac{1}{2} \ln \left(\frac{1 - \varepsilon}{\varepsilon} \right)$$

Adaboost

- **Training exponential loss with Adaboost strictly decreases at every iteration.**

- **Proof:**

- Plug in α , $L_{t+1} = W \cdot 2\sqrt{\epsilon(1-\epsilon)}$
- Since $L_t = W$ $\frac{L_{t+1}}{L_t} = 2\sqrt{\epsilon(1-\epsilon)}$

- If the weak learner is better than random guess, $\epsilon < \frac{1}{2}$,

$$2\sqrt{\epsilon(1-\epsilon)} < 1$$

$$L_{t+1} < L_t$$

Adaboost

- Adaboost reduces bias
- But slightly increases variance

Gradient Boost

- Function optimization using gradient descent, where the function is represented as a sum of weak learners (usually trees).
- And we build F like this:
$$F_m(x) = f_1(x) + f_2(x) + \dots + f_m(x)$$

Algorithm 2 Gradient Boosting

Input: training data $S = (x_1, y_1), \dots, (x_2, y_2), \dots, (x_m, y_m)$

A differential loss function $L(y, F(x))$.

The number of iterations T .

Procedure:

- 1) Initialize the model with a constant value using
$$F_0 = \operatorname{argmin}_{\alpha} \sum_{i=1}^N L(y_i, \alpha)$$
- 2) **for** $m = 1, \dots, M$:
 - (i) Calculate the false residuals
$$r_{mi} = \left[\frac{\delta L(y_i, F(x))}{\delta F(x)} \right]_{F(x)=F_{m-1}(x)} \text{ for } i = 1, \dots, n$$
 - (ii) Train a base learner using the training set
$$D = x_i, r_{mi} \quad i=1, \dots, N$$
 - (iii) Obtain ρ_m by performing the line search optimization:
$$(\rho_m h_m(x)) = \operatorname{argmin}_{\rho, h} \sum_{i=1}^N L(y_i, F_{m-1}(x_i) + \rho h(x_i))$$
 - (iv) Update the model:
$$F_m(x) = F_{m-1}(x) + \rho_m h_m(x)$$

end for

Output: Return the final model $F_m(x)$.

XGBoost

- The XGBoost algorithm is a decision tree-based ensemble that employs the gradient boosting framework
- But adds a regularization term

$$L_M(F(x_i)) = \sum_{i=1}^n L(y_i, F(x_i)) + \sum_{m=1}^M \Omega(h_m)$$
$$\Omega(h) = \gamma T + \frac{1}{2} \lambda ||\omega||^2$$

- The regularization term reduces overfitting

Reference

- <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=9893798>
- <https://hastie.su.domains/TALKS/boost.pdf>
- Optional Reading:
 - The Elements of Statistical Learning, Chapter 9, 10, 15